2024 IEEE Students Conference on Engineering and Systems (SCES), June 21-23, 2024, Prayagraj, India

ReRAM-based Crossbar Compatible Equality Checker for Neuromorphic Applications

Gambali Seshasai Chaitanya
Department of Electronics Engineering
IIT (BHU) Varanasi
Varanasi-005, India
gseshasai.chaitanya.cd.ece22@iitbhu.ac.in

Shubham Pande
Department of Electrical Engineering
IIT Madras
Chennai-036, India
ee18d704@smail.iitm.ac.in

Ankit Arora
Department of Electronics Engineering
IIT (BHU) Varanasi
Varanasi-005, India
ankit.ece@iitbhu.ac.in

***Abstract*—We present our findings on the design and simulation of a ReRAM-based equality checker. The proposed equality checker is compatible with crossbar architectures and is implemented using the Memristive-Aided-Logic (MAGIC) family. The equality checker showcased here is engineered to perform parallel executions such that the number of execution steps remains independent of the number of input bits. Our work provides energy-efficient memristor-based neuromorphic hardware tailored for data-intensive applications.**

Keywords—Memristor, ReRAM, MAGIC, Equality Checker, Neuromorphic Computing

I. Introduction

As machine learning and AI gain widespread adoption, the traditional Von Neumann architecture faces a significant bottleneck in terms of computational performance and energy efficiency [1]. This issue is often attributed to the 'memory wall' problem, which arises from the physical separation of storage and computational units in the Von Neumann architecture. The movement of data consumes more energy and time than the processing itself [1]. Therefore, research efforts are refocusing on exploring Non-Von Neumann architectures implemented on 'beyond-CMOS' platforms with a 'more-than-Moore' approach [2]. Neuromorphic computing, a paradigm that aims to mimic the information-processing principles of the brain, is considered a promising approach to realizing energy-efficient computing systems [3]. The emergence of various non-volatile memory technologies such as Phase Change Memory (PCM), Ferroelectric Field-effect Transistor (FeFET), Magnetic Random Access Memory (MRAM), and Resistive Random Access Memory (ReRAM) has further fueled research activities in the domain of neuromorphic computing [3], [4]. These technologies have shown the potential to overcome the 'memory wall' issue by enabling computation within the memory itself, thus saving significant energy [3], [4]. Resistive Random Access Memory (ReRAM) technology, characterized by its compact two-terminal structure, CMOS compatibility, sub-nanosecond switching speed, and low energy consumption, emerges as a strong contender for in- memory computing-based systems [4]. Furthermore, when ReRAM devices are connected in a specific manner known as a crossbar array, they enable parallel processing of large amounts of data. A typical crossbar architecture, illustrated in Fig. 1, consists of parallel metal lines, referred to as word and bit lines, which intersect perpendicularly. ReRAM cells are placed in the intersections of these word and bit lines, also known as crosspoints. In Fig. 1, the input voltages are applied to the columns, and the total current accumulated from all memristors in a row is collected at one of the ends of the row. The measured output current denotes the weighted sum of the input voltages and ReRAM conductances. Different memristor-based logic families have been reported earlier, such as MAGIC [5], IMPLY [6], Majority Logic [7], and Stateful Logic [8]. Among them, the most popular logic families are MAGIC and IMPLY due to their crossbar compatibility and ease of fabrication. However, the IMPLY logic family utilizes resistors in conjunction with memristors to compute the material implication logic, thus making its crossbar implementation bulky [6]. On the other hand, MAGIC uses only memristors to compute logic, and the output is stored in a separate memristor [5]. Therefore, considering these factors, we choose the MAGIC family for the implementation of the ReRAM-based crossbar-compatible Equality Checker (EC). We demonstrate the functionality of the proposed EC using an experimentally calibrated SPICE-compatible physics-based compact model. The paper is organized as follows: In section II, we discuss the basics of ReRAM, followed by a discussion on the ReRAM compact model. In section III, we discuss the implementation of NOR and NOT logic gates based on the MAGIC family. Section IV discusses the EC implementation, and finally, we conclude in section V.

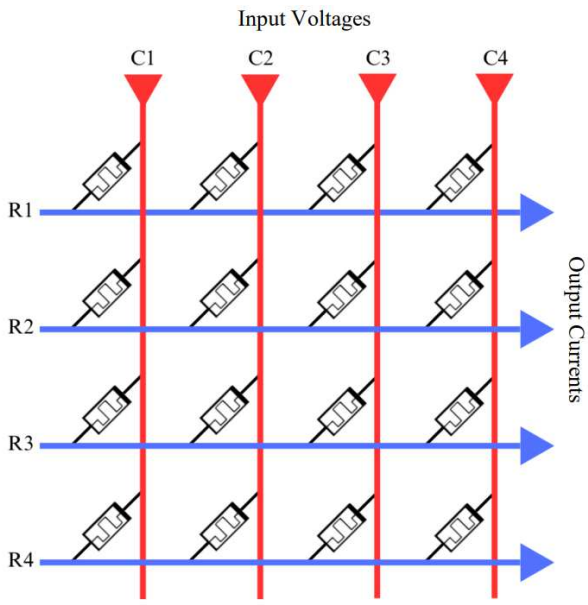

Fig. 1. ReRAM crossbar array schematic showing worldlines, bit lines, and ReRAM at the crosspoints. The input voltages are applied at columns, and the output current is measured in the rows.

II. ReRAM Basics

A Resistive RAM device consists of an insulating layer sandwiched between two metal layers. The metal layer acts as the electrode region, and the insulating layer acts as the resistive switching region. Typical choices of insulator layer

979-8-3503-7471-1/24/$31.00 ©2024 IEEE

include HfO_2, TiO_2, SiO_2, etc[4]. Upon the application of a suitable voltage across electrode terminals, the insulating layer undergoes soft-dielectric breakdown. Subsequently, it is possible to modulate the resistance of the insulating layer by applying the voltage across its terminals. Owing to this phenomenon, a ReRAM device exhibits at least two resistive states – a High Resistance State (HRS) or OFF state that can be used to represent a logical zero, or a Low Resistance State (LRS) or ON state that can be used to represent a logical one [4]. When a voltage pulse of a suitable magnitude is applied across the device in the HRS state, it switches to an LRS state. This occurs due to the formation of conductive paths typically consisting of oxygen vacancies in the insulating layer, as illustrated in Fig. 2 (a). The minimum voltage required to achieve this switching from HRS to LRS is called the SET voltage, denoted by V_{set}. In order to switch the ReRAM device from the Low Resistive State (LRS) to the High Resistive State (HRS), a voltage pulse is applied to the device in the opposite polarity. This voltage is referred to as the RESET voltage, denoted by V_{reset}. During the reset process, the conductive filaments formed during the SET process break down due to the movement of oxygen vacancies driven by electric field and Joule Heating [4]. It is important to note that if the voltage drop across the insulating layer is less than a certain voltage, called threshold voltage, its resistance state does not change. In order to read data from the device, a small read voltage is applied to the device so that it does not disturb the existing resistive state.

A. ReRAM Model Details

The Stanford model is an experimentally calibrated physics-based SPICE-compatible ReRAM model [9]. It models the growth of the conducting filament inside the insulating layer. Specifically, the gap, the distance (g) between the tip of the filament and the opposite electrode, is chosen as the control variable that determines the status (resistance) of the cell. The gap value and the current through the device are modeled effectively with the following equations [9].

$$\frac{d\langle g\rangle}{dt} = -v_0 \times exp\left(-\frac{E_A}{kT}\right) \times sinh\left(\gamma \times \frac{a_0}{t_{OX}} \times \frac{qV}{kT}\right) \quad (1)$$

$$I = I_0 \times exp\left(-\frac{g}{g_0}\right) \times sinh\left(\frac{V}{V_0}\right) \quad (2)$$

We use the Stanford model for the SPICE simulations of the proposed equality checker. The simulated DC current-voltage characteristics of ReRAM are shown in Fig. 2(b). Fig. 2(c) the pulse-switching characteristics of ReRAM. The region highlighted in green represents the set process - transition from the HRS to the LRS. The region highlighted in yellow represents the reset process - the transition from the LRS to the HRS. For more details about the model and model parameters, we refer the reader to [9].

III. Logic Gates Using Magic Logic

In this section, we discuss the implementation of NOR and NOT logic gates, which are further used in the implementation of the equality checker. These logic gates are implemented using a logic family designed for crossbar arrays of ReRAM devices, called MAGIC—Memristor-Aided Logic [5].

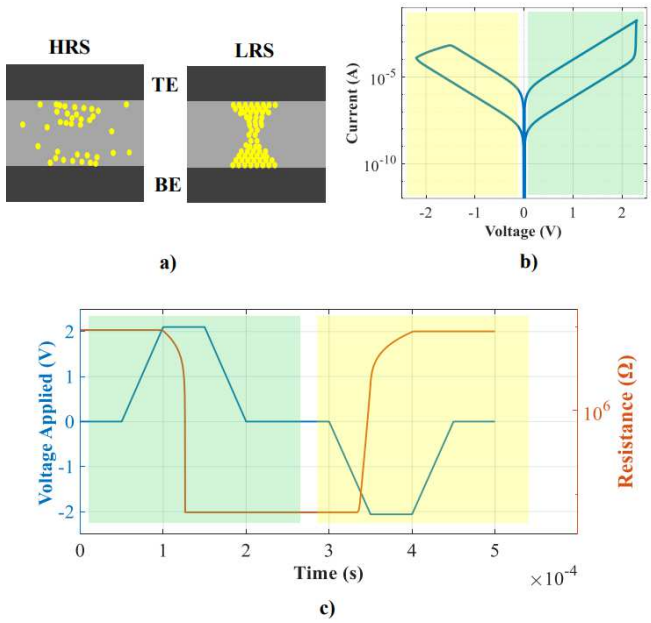

Fig. 2. (a) Conductive filament (CF) dissolution and formation ReRAM. The CF consists of oxygen vacancies, as shown by the yellow circles. (b) DC current-voltage characteristics of ReRAM (c) Evolution of ReRAM resistance set (shaded in green color) and reset (shaded in yellow color) process.

A. MAGIC NOT Gate

The series connection of two memristors with opposite polarity enables the logical NOT operation, as shown in Fig.3(a). The input is shown in blue, and the output memristor is shown in red color, respectively. Note that prior to the execution of the logical operation, the output memristor must be programmed to the LRS. This is achieved by applying an appropriate voltage at crossbar rows and columns corresponding to each memristor cell and using standard read-write-verify ReRAM programming schemes [3]–[5]. Fig.2(b) illustrates the operation of the NOT gate, with logical 1 and logical 0 states represented by ReRAM in the LRS and HRS, respectively. The specific values of LRS and HRS used in the simulation are 61 kΩ and 67 MΩ, respectively. When the input ReRAM is set to LRS, the voltage drop across the output ReRAM exceeds its threshold voltage. Consequently, the output ReRAM resistance changes from LRS to HRS state, as depicted by solid lines. Conversely, when the input is in HRS, most of the applied voltage is dropped across the input ReRAM, hence, the output ReRAM remains in LRS, indicated by the dashed line. Thus, the series combination of two ReRAM cells realizes the NOT operation.

B. MAGIC NOR Gate

A ReRAM-based 2-input NOR gate implementation is illustrated in Fig. 4(a). The input ReRAMs are depicted in blue, while the output ReRAM is shown in red. Initially, the output memristor is initialized to the LRS using standard programming schemes [3]–[5]. During the logic execution step, a voltage V0 is applied to the input ReRAMs. In the case where both input ReRAMs are in the HRS, i.e., logical 0, the voltage drop across the output ReRAM will be less than its threshold voltage. Consequently, its logical state remains unchanged, representing a logical 1 (LRS). For all other input combinations, the voltage drop across the output ReRAM exceeds its threshold, thus changing its logical state, as

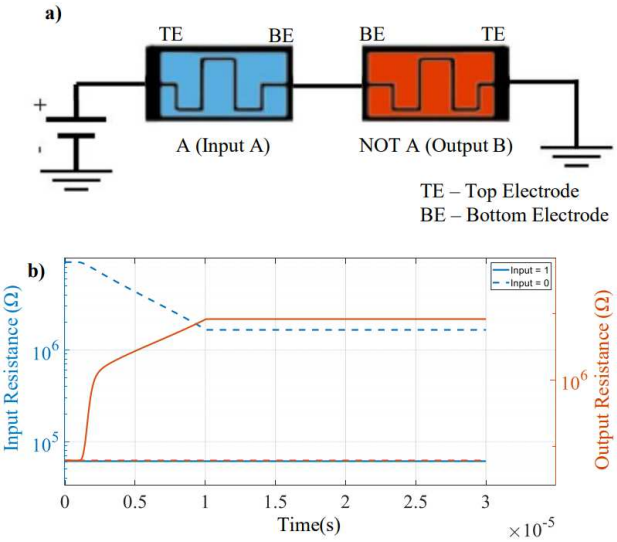

Fig. 3. (a) MAGIC NOT gate schematic (b) Operation of the NOT gate. The input ReRAM resistance is depicted in blue, while the output ReRAM resistance is represented in red. When the input is logical 1, the output ReRAM changes its resistance from logical 1 to logical 0, as shown by the solid line. Output ReRAM resistance remains in the logical 1 state when the input ReRAM is in the logical 0 state, as shown by the dashed line.

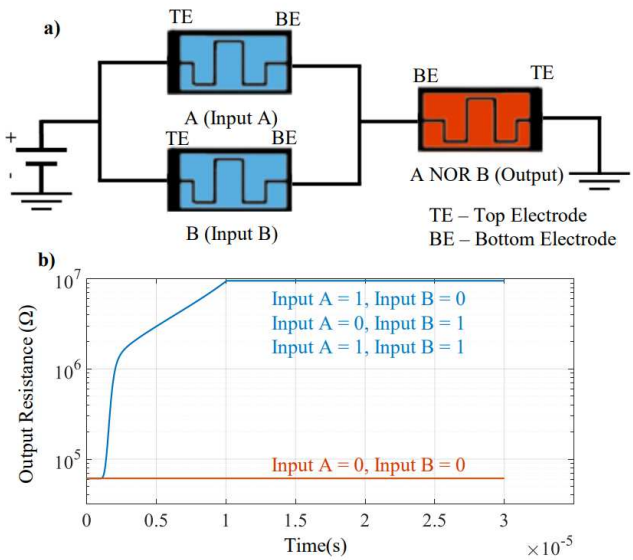

Fig. 4. (a) MAGIC NOR gate schematic (b) Operation of NOT gate: Output ReRAM resistance evolution for four possible input combinations

illustrated in Fig. 4(b). For example, for the input combination of A = 0 & B = 1, A = 1 & B = 0, A = 1 & B = 1, i.e., when one of the input ReRAMs is set to LRS, the resistance of the output ReRAM transitions from LRS to HRS. Conversely, for the input combination A = 0 & B = 0, indicating that both input ReRAMs are in HRS, the output ReRAM remains in LRS.

IV. Equality Checker

An equality checker accepts two n-bit keys and checks them for equality. It returns "1" if both the bit-strings being compared are equal or "0" otherwise. Both the bit strings that are being compared for equality are stored vertically in the first and second columns, as shown in Fig. 5(a). In the proposed design, all the bits of the search string and the stored data string are processed parallelly. The detailed working of the equality checker is discussed in the following sections.

A. Initialization

Before conducting an equality check, it is necessary to write the data to be compared onto the first and second columns where the input is stored. This is accomplished by applying appropriate voltages at the crossbar rows and columns corresponding to each memristor cell, utilizing one of the standard read-write-verify ReRAM programming schemes [3]–[5]. Additionally, all other cells must be initialized to a logical one or low-resistance state (LRS) as required by the MAGIC logic family [5].

B. Equality Check Operation

Let A_i and B_i represent the ith bit of the of the strings. In the first step, we compute A_i NOR B_i, labelled as K_i, followed by A_i NOR (A_i NOR B_i) - labelled as X_i, and B_i NOR (A_i NOR B_i) - labelled as Y_i, in the subsequent steps. Next, we evaluate X_i NOR Y_i, effectively yielding A_i XNOR B_i. Subsequently, we invert the obtained bits using a NOT gate and store them in the last column (Step 5). Finally, we perform the NOR operation over the column to obtain the result. In order to prevent unselected ReRAM cells from changing their resistive state, an isolation voltage, $V_{iso} = V_0/2$ is applied across their rows and columns [5], where V_0 is execution voltage applied across the selected ReRAM. An n-bit equality checker necessitates a crossbar array of dimensions $(n + 1) \times 7$. The equality checking for all the bits is performed in parallel; therefore, it requires the same number of execution steps, i.e., 6, regardless of the length of the strings. For a practical demonstration of the proposed equality checker circuit, a simple 2-bit version is simulated, and step-by-step outputs are illustrated in Fig. 5 (b)-(g). In this simulation, a two-bit binary number, 10, is stored in the first column, and 11 is stored in the second column. Consequently, A_0 equals 1, A_1 equals 0, and B_0 equals B_1 equals 1. The stepwise description of the operation executed is provided below.

1) First, we perform A_i NOR B_i, which gives us $K_0 = 0$ and $K_1 = 0$, as shown in Fig. 5(b)
2) A_i NOR K_i, which gives us $X_0 = 0$ and $X_0 = 1$, as shown in Fig. 5(c)
3) B_i NOR K_i, which gives us $Y_0 = 0$ and $Y_1 = 0$, as shown in Fig. 5(d)
4) X_i NOR Y_i, which gives us $Z_0 = 1$ and $Z_1 = 0$, as shown in Fig. 5(e)
5) $\overline{Z}_i$ = NOT Z_i which gives us $\overline{Z}_0 = 0$ and $\overline{Z}_1 = 1$ as shown in Fig. 5(f). We have now essentially calculated A_i XOR B_i.

In the sixth and final step, we compute the NOR of $\overline{Z}_0$ and $\overline{Z}_1$ to obtain the final output, which is logical 0 as shown in Fig. 5(g). This result indicates that the two strings being compared are not equal, as demonstrated in Fig. 5 (g). The NOR operation within a column is performed using the transpose memory scheme in MAGIC Logic [10]. Execution within a column differs slightly from row execution. In this scheme, the input memristors are connected to the ground, and the execution voltage V_0 is applied to the output memristor. The execution voltages required at each step are listed in table I.

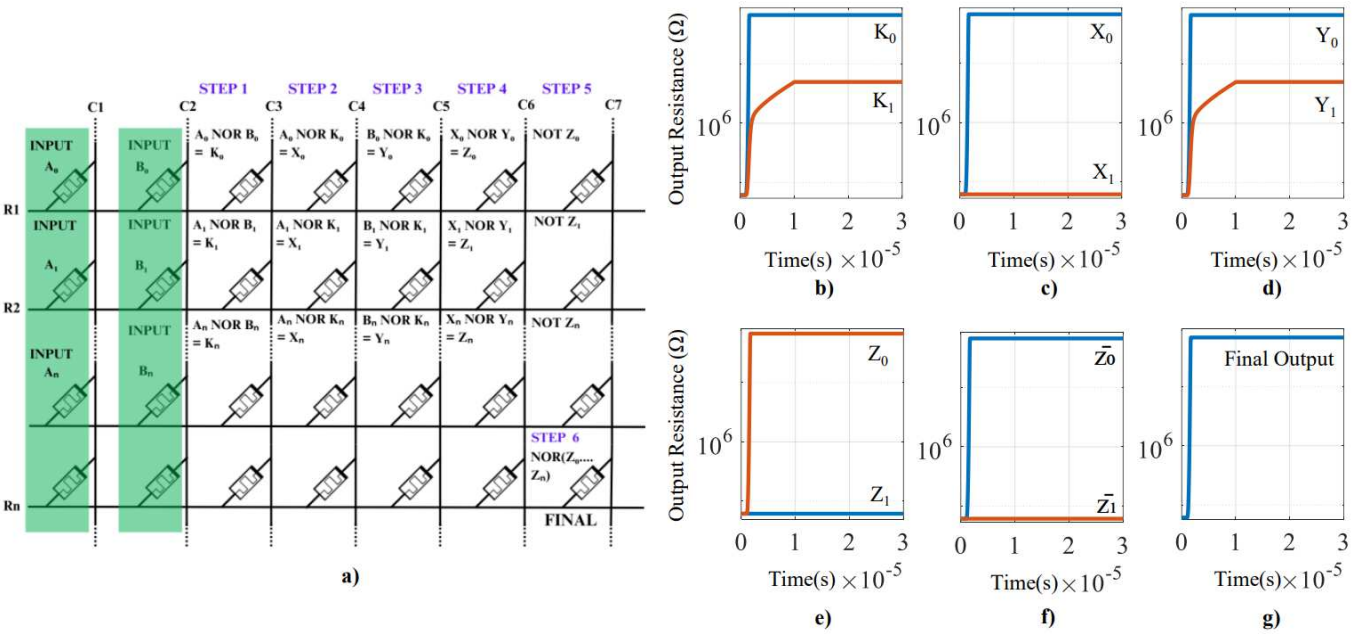

Fig. 5. (a) Schematic of the equality checker (b-g) ReRAM resistance evolution in each step of equality check operation.

TABLE I.

Step	Voltages (V_0 = 3.22V, V_{iso} = V_0/2)	Outputs (i = 1...n)
1	C1 = C2 = V_0, C3 = 0, R9 = V_{iso}	NOR (A_i, B_i) = K_i
2	C1 = C3 = V_0, C3 = 0, R9 = V_{iso}	NOR (A_i, K_i) = X_i
3	C2 = C3 = V_0, C5 = 0, R9 = V_{iso}	NOR (B_i, K_i) = Y_i
4	C4 = C5 = V_0, C6 = 0, R9 = V_{iso}	NOR (X_i, Y_i) = Z_i
5	C6 = V_0, C7 = 0, R9 = V_{iso}	$\overline{Z}_i$ = NOT (Z_i)
6	R9 = V_0 R1 = R2 = R3 = R4 = V_{iso} R5 = R6 = R7 = R8 = V_{iso} C1 = C2 = C3 = C4 = C5 = C6 = V_{iso}	FINAL = NOR ($\overline{Z}_i$, i = 1 to n)

V. Conclusion

We demonstrated a novel Equality Checker circuit that is based on a purely memristive 0T-1R crossbar array architecture implemented using the MAGIC logic family. An n-bit equality checker requires a crossbar array with $(n + 1) \times 7$ memristor devices. The reported design performs all the equality checking operations in parallel. Regardless of the length of the bit strings being checked for equality, the circuit requires precisely 6 execution steps (each execution step corresponds to a different set of row and column voltages) to obtain the final result. Therefore, it can be extended to be used for bit strings of any length without any change in the execution time, proving it to be highly scalable and very suitable for data-intensive applications.

References

[1] Sally A. Mckee. Reflections on the memory wall. In CF '04, 2004.

[2] Suman Datta, Wriddhi Chakraborty, and Marko Radosavljevic. Toward attojoule switching energy in logic transistors. Science, 378(6621):733– 740, 2022.

[3] J. Joshua Yang, Dmitri B. Strukov, and Duncan R. Stewart. Memristive devices for computing. Nature nanotechnology, 8 1:13–24, 2013.

[4] Furqan Zahoor, Tun Zainal Azni Zulkifli, and Farooq Ahmad Khanday. Resistive random access memory (rram): an overview of materials, switching mechanism, performance, multilevel cell (mlc) storage, modeling, and applications. Nanoscale Research Letters, 15, 2020.

[5] Shahar Kvatinsky, Dmitry Belousov, Slavik Liman, Guy Satat, Nimrod Wald, Eby G. Friedman, Avinoam Kolodny, and Uri C. Weiser. Magic—memristor-aided logic. IEEE Transactions on Circuits and Systems II: Express Briefs, 61:895–899, 2014.

[6] Shahar Kvatinsky, Guy Satat, Nimrod Wald, Eby G. Friedman, Avinoam Kolodny, and Uri C. Weiser. Memristor-based material implication (imply) logic: Design principles and methodologies. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 22(10):2054–2066, 2014.

[7] John Reuben. Rediscovering majority logic in the post-cmos era: A perspective from in-memory computing. Journal of Low Power Electronics and Applications, 2020.

[8] Eero Lehtonen and Mika Laiho. Stateful implication logic with memristors. In 2009 IEEE/ACM International Symposium on Nanoscale Architectures, pages 33–36, 2009.

[9] Zizhen Jiang, Yi Wu, Shimeng Yu, Lin Yang, Kay Song, Zia Karim, and H.-S. Philip Wong. A compact model for metal–oxide resistive random access memory with experiment verification. IEEE Transactions on Electron Devices, 63(5):1884–1892, 2016.

[10] Nishil Talati, Saransh Gupta, Pravin Mane, and Shahar Kvatinsky. Logic design within memristive memories using memristor-aided logic (magic). IEEE Transactions on Nanotechnology, 15(4):635–650, 2016.